

Software Development Process and Team Task Sheet

Project Overview

The Rental Property Management Software is a full-stack web application for managing rental rooms, tenants, lease contracts, invoices, payments, maintenance requests, notifications, financial reports, legal reports, and audit logs.

The repository is organized into two main applications:

- backend : Node.js, Express, SQLite, JWT authentication, REST APIs, business services, middleware, and Jest tests.
- frontend : React, Vite, protected routes, landlord pages, tenant pages, reusable layout components, and API integration.

The project was developed through five sprints. Each sprint focused on a clear set of user-facing and technical goals, and responsibilities were divided among team members according to their roles.

Team Members

Name	Role
Đoàn Quốc Kiên	Team Leader / Full-stack Developer
Trần Vĩnh Khang	Back-end Developer
Nguyễn Khôi Nguyên	Front-end Developer
Đặng Võ Tuấn Tài	Front-end Developer / QA Tester

Sprint 1: Foundation, Authentication, and Room Management

Sprint duration: 08/04/2026 - 13/04/2026

Sprint goal: Build the core technical foundation of the system, including project structure, database initialization, basic authentication, and room management for landlords.

Sprint 1 Task Sheet

Name	Task Title	Task Description	Progress	Rate of Success
Đoàn Quốc Kiên	Project foundation and authentication setup	Initialized the repository, configured the backend and frontend base structure, created the initial database schema, and implemented login/logout APIs.	Met deadline	100%
Trần Vĩnh Khang	Room management backend APIs	Developed room CRUD APIs including create, list, update, delete, and filtering by room status or category.	Met deadline	100%
Nguyễn Hoài Nguyên	Main UI layout and authentication interface	Built the main frontend layout, responsive page structure, Vietnamese UI support, browser compatibility, and login screen integration.	Met deadline	100%
Đặng Võ Tuấn Tài	Room management UI and QA	Built the landlord room list screen, add/edit room form, and tested the room creation flow and response time.	Met deadline	100%

Detailed Task Description

Đoàn Quốc Kiên was responsible for setting up the technical foundation of the project. This included creating the initial repository structure, organizing the backend and frontend folders, configuring the development environment, and documenting how to install and run the project. On the backend side, the database schema was initialized in `backend/database.js` , including core entities such as users and rooms. Authentication work was implemented through routes and middleware such as `backend/routes/auth.js` , `backend/middleware/auth.js` , and `backend/config/auth.js` , allowing users to log in, log out, and access protected APIs securely.

Trần Vĩnh Khang focused on the room management backend. The main result of this work is visible in `backend/routes/rooms.js` , where the system supports creating rooms, retrieving the landlord's room list, filtering rooms, updating room details, and deleting rooms. The APIs validate important room information such as room name, price, area, maximum occupants, category, and status. These APIs form the foundation for later contract and billing workflows because contracts and invoices depend on accurate room data.

Nguyễn Khôi Nguyên worked on the initial frontend layout and authentication interface. This included setting up React pages, reusable layout components, protected route behavior, and Vietnamese-friendly UI content. Relevant files include `frontend/src/App.jsx` , `frontend/src/components/layout/MainLayout.jsx` , `frontend/src/components/layout/Sidebar.jsx` , `frontend/src/pages/Login.jsx` , and supporting frontend context files. The work ensured that the application had a usable base interface for both landlord and tenant workflows.

Đặng Võ Tuấn Tài built the landlord-facing room management interface and performed QA for Sprint 1. This included implementing room list display, add/edit room forms, and checking whether landlords could add a room successfully. Related frontend work is represented in files such as `frontend/src/components/RoomForm.jsx` , `frontend/src/pages/Overview.jsx` , and `frontend/src/pages/LandlordDashboard.jsx` . QA focused on making sure the interface was easy to use, responsive, and fast enough for the project requirement.

Sprint 2: Tenant Profile and Contract Management

Sprint duration: 14/04/2026 - 17/04/2026

Sprint goal: Connect tenants to the system by allowing tenant registration, landlord approval, tenant profile management, and lease contract creation.

Sprint 2 Task Sheet

Name	Task Title	Task Description	Progress	Rate of Success
Đoàn Quốc Kiên	Deployment setup and contract creation API	Configured deployment workflow, implemented core contract creation logic, and ensured room status changes to occupied after contract creation.	Met deadline	100%

Name	Task Title	Task Description	Progress	Rate of Success
Trần Vĩnh Khang	Tenant and contract backend APIs	Developed tenant CRUD APIs, account approval APIs, and contract detail retrieval APIs.	Met deadline	100%
Nguyễn Khôi Nguyễn	Tenant portal profile and contract views	Built tenant registration, tenant profile update, and tenant contract detail interfaces.	Met deadline	100%
Đặng Võ Tuấn Tài	Landlord tenant management and contract form	Built landlord tenant list, pending approval UI, contract creation form, and tested contract workflows.	Met deadline	100%

Detailed Task Description

Đoàn Quốc Kiên handled the most important backend logic of Sprint 2: creating lease contracts. The contract API is implemented in `backend/routes/contracts.js` . It validates room ID, tenant ID, start date, end date, deposit, rental price, and utility prices. It also ensures that a contract can only be created for an available room and a valid tenant account. When the contract is successfully created, the room status is automatically changed to `occupied` , which keeps the room inventory consistent with the contract data.

Trần Vĩnh Khang implemented backend tenant management. The main API work is in `backend/routes/tenants.js` , where landlords can manage tenants, approve or update tenant status, retrieve tenant lists, and view tenant details. This sprint also included APIs for contract details so that both landlords and tenants could view relevant contract information. These APIs are important because they connect account management, room assignment, and future billing workflows.

Nguyễn Khôi Nguyễn developed tenant-facing frontend pages. This work includes tenant registration, profile editing, and contract viewing. Relevant frontend files include `frontend/src/pages/Register.jsx` , `frontend/src/pages/tenant/TenantProfile.jsx` , `frontend/src/pages/tenant/TenantContract.jsx` , and `frontend/src/pages/ContractDetail.jsx` . The goal was to let tenants provide personal information such as full name, phone number, citizen ID, date of birth, gender, and address so the landlord could create valid lease contracts.

Đặng Võ Tuấn Tài worked on landlord-facing tenant management and contract creation screens. The related pages include `frontend/src/pages/Tenants.jsx` , `frontend/src/pages/TenantApproval.jsx` , and

frontend/src/pages/Contract.jsx . QA activities focused on confirming that tenant approval worked correctly, that contract forms contained the required fields, and that contract creation correctly changed the room status.

Sprint 3: Invoices, Meter Readings, and Maintenance

Sprint duration: 19/04/2026 - 26/04/2026

Sprint goal: Implement utility reading, automatic invoice calculation, tenant maintenance requests, and manager maintenance processing.

Sprint 3 Task Sheet

Name	Task Title	Task Description	Progress	Rate of Success
Đoàn Quốc Kiên	Invoice calculation service and validation	Implemented invoice calculation logic, fault tolerance, missing price validation, and review of previous meter reading logic.	Met deadline	100%
Trần Vĩnh Khang	Meter reading, invoice, and maintenance APIs	Developed APIs for meter readings, invoice creation, maintenance request CRUD, file upload, and maintenance status updates.	Met deadline	100%
Nguyễn Hoài Nguyên	Tenant invoices and maintenance UI	Built tenant invoice history and tenant maintenance request form with validation and image upload flow.	Met deadline	100%
Đặng Võ Tuấn Tài	Manager utility and maintenance UI with QA	Built manager meter reading screen, manager maintenance screen, and tested invoice calculation accuracy.	Met deadline	100%

Detailed Task Description

Đoàn Quốc Kiên implemented the core billing calculation logic in `backend/services/invoiceCalculator.js`. This service calculates the invoice total using room price, previous and current electricity readings, previous and current water readings, utility unit prices, and service fees such as WiFi and trash. It also validates invalid billing data, prevents decreasing meter indexes, and throws clear errors when electricity or water unit prices are missing. This improves fault tolerance by preventing the system from generating incorrect invoices.

Trần Vĩnh Khang developed backend APIs for utility readings, invoices, and maintenance. Meter reading logic is implemented in `backend/routes/meter-readings.js`, while invoice-related APIs are implemented in `backend/routes/invoices.js` and landlord billing support is implemented in `backend/routes/landlord.js`. Maintenance request APIs are implemented in `backend/routes/maintenance-requests.js`, including request creation, status updates, manager assignment, resolution notes, and file upload support through `backend/middleware/upload.js`.

Nguyễn Khôi Nguyên worked on tenant-facing invoice and maintenance pages. The tenant invoice history is represented by `frontend/src/pages/tenant/TenantInvoices.jsx`, and tenant maintenance features are represented by `frontend/src/pages/tenant/TenantMaintenance.jsx`. These pages allow tenants to view billing information and submit maintenance issues with descriptions and images. The UI validates required input so empty maintenance reports are not submitted.

Đặng Võ Tuấn Tài built manager-facing utility and maintenance interfaces. Relevant files include `frontend/src/pages/ManagerMeterReading.jsx` and `frontend/src/pages/ManagerMaintenance.jsx`. QA for this sprint focused especially on the invoice calculation formula:
 $(\text{new index} - \text{old index}) * \text{unit price} + \text{room price} + \text{service fees}$. The expected result was compared manually with calculator results to ensure billing correctness.

Sprint 4: Online Payment, Financial Dashboard, and Notifications

Sprint duration: 30/04/2026 - 07/05/2026

Sprint goal: Support online payment simulation, financial statistics, bad debt tracking, and notification workflows for tenants and landlords.

Sprint 4 Task Sheet

Name	Task Title	Task Description	Progress	Rate of Success
Đoàn Quốc Kiên	Payment integration and automated status update	Implemented payment flow logic, mock payment confirmation, invoice payment status update, and foundation for automatic reminders.	Met deadline	100%
Trần Vĩnh Khang	Financial statistics and notification APIs	Developed financial aggregation APIs and notification APIs for landlord broadcast messages.	Met deadline	100%
Nguyễn Khôi Nguyên	Tenant payment flow and notification inbox	Built tenant invoice payment UI, mock QR payment screen, and notification reading interface.	Met deadline	100%
Đặng Võ Tuấn Tài	Financial dashboard, notification form, and QA	Built owner financial dashboard, broadcast notification UI, and tested payment happy and alternative flows.	Met deadline	100%

Detailed Task Description

Đoàn Quốc Kiên worked on the payment workflow and automatic invoice status updates. In this repository, payment simulation is visible in `backend/routes/invoices.js`, especially the mock payment endpoints that retrieve invoice payment data and confirm payments. These APIs calculate paid amount, remaining amount, partial payment status, and full payment status. This provides a realistic foundation for later integration with real payment gateways such as VNPAY, Momo, PayOS, or bank QR systems.

Trần Vĩnh Khang implemented backend support for financial statistics and notifications. Financial dashboard data is supported through landlord/invoice/room APIs such as `backend/routes/landlord.js`, `backend/routes/invoices.js`, and `backend/routes/rooms.js`. Notification APIs are implemented in `backend/routes/notifications.js`, supporting landlord-created announcements, recipient targeting, unread count, read status, and deletion.

Nguyễn Khôi Nguyên built tenant-facing payment and notification screens. Relevant frontend files include `frontend/src/pages/tenant/TenantInvoices.jsx`, `frontend/src/pages/MockPayment.jsx`,

`frontend/src/pages/TenantNotifications.jsx` , and `frontend/src/components/NotificationBell.jsx` . These pages allow tenants to view unpaid invoices, open a mock payment screen, confirm payment behavior, and read notifications from the landlord.

Đặng Võ Tuấn Tài implemented the landlord financial dashboard and notification UI. The dashboard is represented by `frontend/src/pages/FinancialDashboard.jsx` , using charting and summary statistics to show revenue, occupancy, and bad debts. Broadcast notification UI is represented by `frontend/src/pages/NotificationSystem.jsx` . QA testing included successful payment, partial/alternative payment behavior, notification creation, validation messages, and dashboard data display.

Sprint 5: Reports, Audit Log, Settings, and Robustness

Sprint duration: 10/05/2026 - 17/05/2026

Sprint goal: Complete legal and tax reporting, audit logging, service configuration UI, notification robustness, and improved error handling.

Sprint 5 Task Sheet

Name	Task Title	Task Description	Progress	Rate of Success
Đoàn Quốc Kiên	Audit logging middleware and test suite support	Implemented audit logging middleware for sensitive updates/deletes, audit-log database compatibility, and backend service/API tests.	Met deadline	100%
Trần Vĩnh Khang	Legal and tax report APIs	Developed temporary residence PDF report API and tax report export API using tenant, contract, and paid invoice data.	Met deadline	100%
Nguyễn Khôi Nguyên	Settings and notification UI	Built settings/configuration interface and notification components for automatic alerts and polling-based updates.	Met deadline	100%

Name	Task Title	Task Description	Progress	Rate of Success
Đặng Võ Tuấn Tài	Audit log UI, report downloads, and error handling	Built audit log screen, report download behavior, and payment/invoice error handling improvements.	Met deadline	100%

Detailed Task Description

Đoàn Quốc Kiên implemented audit logging support for sensitive system changes. The middleware in `backend/middleware/audit.js` intercepts update and delete operations for selected resources such as rooms and lease contracts, records the user ID, action type, target table, target ID, request path, HTTP method, payload, response status, and timestamp. Database compatibility for audit logs is maintained in `backend/database.js`, while audit log service functions are implemented in `backend/services/auditLog.js`. Backend test support is represented by `backend/__tests__/api.test.js` and `backend/__tests__/services.test.js`, which verify API behavior and service logic.

Trần Vĩnh Khang developed legal and tax report APIs in `backend/routes/reports.js`. The temporary residence report API retrieves active tenant and contract data, then exports it into a PDF document. The tax report API retrieves paid invoices, calculates revenue totals, and exports the result through Excel-compatible output. These reports support legal residence registration and financial/tax declaration needs for landlords.

Nguyễn Khôi Nguyên worked on configuration and notification-related UI. The settings screen is represented by `frontend/src/pages/Settings.jsx`, and notification behavior is represented by `frontend/src/components/NotificationBell.jsx`, `frontend/src/pages/TenantNotifications.jsx`, and related notification pages. This work supports landlord configuration and gives tenants a clearer way to receive system alerts.

Đặng Võ Tuấn Tài implemented frontend audit and report workflows. The audit log interface is represented by `frontend/src/pages/AuditLog.jsx`, where landlords can review system operation history and filter logs. Report download behavior connects frontend actions to backend report APIs, allowing downloaded files to be handled correctly by the browser. Robustness improvements focused on showing meaningful errors instead of allowing payment, invoice, or report failures to crash the interface.

Repository Evidence of Sprint Outcomes

Feature Area	Representative Files
Authentication	backend/routes/auth.js , backend/middleware/auth.js , frontend/src/context/AuthContext.jsx , frontend/src/pages/Login.jsx , frontend/src/pages/Register.jsx
Room Management	backend/routes/rooms.js , frontend/src/components/RoomForm.jsx , frontend/src/pages/LandlordDashboard.jsx , frontend/src/pages/Overview.jsx
Tenant Management	backend/routes/tenants.js , frontend/src/pages/Tenants.jsx , frontend/src/pages/TenantApproval.jsx , frontend/src/pages/tenant/TenantProfile.jsx
Contract Management	backend/routes/contracts.js , frontend/src/pages/Contract.jsx , frontend/src/pages/ContractDetail.jsx , frontend/src/pages/tenant/TenantContract.jsx
Billing and Meter Reading	backend/services/invoiceCalculator.js , backend/services/meterReadingService.js , backend/routes/meter-readings.js , backend/routes/invoices.js , backend/routes/landlord.js , frontend/src/pages/Invoices.jsx , frontend/src/pages/ManagerMeterReading.jsx
Maintenance	backend/routes/maintenance-requests.js , backend/middleware/upload.js , frontend/src/pages/ManagerMaintenance.jsx , frontend/src/pages/tenant/TenantMaintenance.jsx
Payments	backend/routes/invoices.js , frontend/src/pages/MockPayment.jsx , frontend/src/pages/tenant/TenantInvoices.jsx
Financial Dashboard	frontend/src/pages/FinancialDashboard.jsx , backend/routes/landlord.js , backend/routes/invoices.js
Notifications	backend/routes/notifications.js , frontend/src/pages/NotificationSystem.jsx , frontend/src/pages/TenantNotifications.jsx , frontend/src/components/NotificationBell.jsx

Feature Area	Representative Files
Reports	backend/routes/reports.js , backend/fonts/arial.ttf , backend/fonts/arialbd.ttf
Audit Log	backend/middleware/audit.js , backend/services/auditLog.js , backend/routes/audit.js , frontend/src/pages/AuditLog.jsx
Testing	backend/__tests__/api.test.js , backend/__tests__/services.test.js , TEST_CASE_SUITE_REPORT.md

Overall Development Process

The team followed an iterative and incremental development process. Instead of attempting to build the entire rental management system at once, the project was divided into small sprint goals. Each sprint produced a usable set of features that could be reviewed, tested, and extended in the next sprint.

The development process followed these practices:

- Sprint planning was used to define the sprint goal, backlog items, and task ownership.
- Backend and frontend work were developed in parallel after API responsibilities were clarified.
- Database schema updates were implemented gradually as new features required new entities or columns.
- Role-based access control was applied to protect landlord and tenant workflows.
- Each sprint included testing activities, especially for business-critical flows such as authentication, room creation, contract creation, invoice calculation, payment status updates, and audit logging.
- The final sprint added automated Jest tests and coverage reporting to improve maintainability and confidence.

Software Development Life Cycle (SDLC)

The project followed the Software Development Life Cycle through the following phases.

1. Requirement Analysis

The team identified the main users of the system: landlords, tenants, managers, and technical staff. Functional requirements were grouped into use cases such as authentication, room management,

tenant management, contract management, invoice generation, maintenance handling, payment, notification, reporting, and audit logging.

During this phase, the team clarified what each role needed:

- Landlords need to manage rooms, tenants, contracts, invoices, payments, reports, and notifications.
- Tenants need to register, update profiles, view contracts, view invoices, pay invoices, receive notifications, and submit maintenance requests.
- Managers need to track utility readings and process maintenance operations.
- The system needs secure authentication, reliable data storage, and role-based access control.

2. Planning

The project was divided into five sprints. Each sprint had a defined goal, task backlog, responsible team members, and completion criteria. This planning approach allowed the team to focus on one group of related features at a time.

The sprint order was logical:

- Sprint 1 created the technical foundation and room management.
- Sprint 2 added tenants and contracts.
- Sprint 3 added billing and maintenance.
- Sprint 4 added payment, finance, and notifications.
- Sprint 5 added reports, audit logging, configuration, robustness, and testing.

3. System Design

The system was designed as a full-stack web application. The frontend communicates with the backend through REST APIs. The backend uses Express routes to separate feature areas, and SQLite stores persistent data.

Important design decisions include:

- JWT authentication for secure login and protected routes.
- Refresh token storage and revoked access token tracking for better session control.
- Separate route files for each domain, such as rooms, tenants, contracts, invoices, notifications, reports, and audit logs.
- Service modules for reusable business logic, such as invoice calculation and previous meter reading lookup.
- Middleware for authentication, upload handling, and audit logging.

- React page components organized by landlord and tenant workflows.

4. Implementation

Implementation was completed sprint by sprint. Backend developers created database tables, route handlers, validation rules, and business services. Frontend developers built responsive pages, forms, dashboards, and tenant/landlord workflows. QA activities were performed throughout each sprint to verify expected behavior.

The implementation phase produced working modules for:

- Authentication and account registration.
- Room CRUD and availability tracking.
- Tenant profiles and landlord approval.
- Lease contract creation and room status synchronization.
- Meter readings and invoice generation.
- Maintenance request submission and management.
- Mock payment and invoice payment status updates.
- Financial dashboard and notification system.
- Legal/tax report export.
- Audit log tracking and display.

5. Testing

Testing was performed through both manual and automated methods. Manual testing was used during each sprint to verify UI flows, API behavior, form validation, and business rules. Automated testing was added for backend APIs and service logic.

Automated tests are located in:

- `backend/__tests__/api.test.js`
- `backend/__tests__/services.test.js`

The latest automated test result:

```
Test Suites: 2 passed, 2 total
Tests:      49 passed, 49 total
```

Coverage reporting is available through:

```
npm.cmd run test:coverage
```

6. Deployment and Integration

The project structure supports separate backend and frontend execution. The backend runs as an Express server, while the frontend runs through Vite. The deployment plan includes hosting the frontend on a web platform such as Vercel and deploying the backend to a Node.js-compatible hosting environment.

Integration work focused on connecting React pages to backend APIs, protecting routes by role, and ensuring that frontend actions update backend state correctly.

7. Maintenance and Improvement

Maintenance work includes improving validation, adding new reports, increasing automated test coverage, fixing discovered issues, and preparing the system for production deployment. The audit logging feature supports maintenance by making sensitive changes traceable. The test suite and coverage command also support future maintenance by helping developers detect regressions earlier.

Conclusion

Across five sprints, the team developed the Rental Property Management Software from a basic project foundation into a functional full-stack system. The project now includes landlord and tenant workflows, contract management, invoice calculation, maintenance handling, payment simulation, financial dashboards, notifications, reports, audit logging, and automated tests.

All sprint tasks were completed on time according to the provided sprint records, and each team member achieved a 100% success rate for assigned work.